***BRACT's* Vishwakarma Institute of Technology, Pune**

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No:3

Roll no: 31 PRN No:12414893 Name of Student: Kartiki Vinayak Rasane

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

1. Import the required TensorFlow/Keras and preprocessing libraries.
2. Load the dataset and split it into training and testing sets.
3. Normalize the input features using StandardScaler.
4. Build a neural network with input, hidden, and output layers.
5. Compile the model using the SGD optimizer with a chosen learning rate.
6. Train the model using forward propagation and backpropagation for a specified number of epochs.
7. Evaluate the model on the test dataset.
8. Repeat the training with different learning rates and epochs.
9. Compare the accuracy and loss to analyze model performance.

Code & Output:

[1] ✓ 7s

```python
# Import libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

[2] ✓ 0s

```python
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
```

[3] ✓ 0s

```python
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[4] ✓ 0s

```python
# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```python
# Function to create model
def create_model(learning_rate):
    model = Sequential([
        Dense(10, activation='relu', input_shape=(4,)),
        Dense(8, activation='relu'),
        Dense(3, activation='softmax')
    ])

    optimizer = SGD(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# Different learning rates
learning_rates = [0.001, 0.01, 0.1]

# Different epochs
epochs_list = [20, 50, 100]

results = {}

for lr in learning_rates:
    for epochs in epochs_list:
        print(f"\nTraining with Learning Rate={lr}, Epochs={epochs}")

        model = create_model(lr)

        history = model.fit(
```

```
Training with Learning Rate=0.01, Epochs=100
Test Accuracy: 0.9333

Training with Learning Rate=0.1, Epochs=20
Test Accuracy: 1.0

Training with Learning Rate=0.1, Epochs=50
Test Accuracy: 0.9667

Training with Learning Rate=0.1, Epochs=100
Test Accuracy: 1.0
```

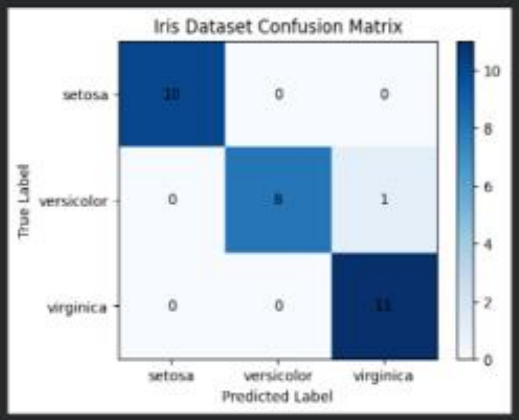
```python
# Plot accuracy for one example
history = results[(0.01, 50)]['history']

plt.plot(history['accuracy'], label='Training Accuracy')
plt.plot(history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()

# Print summary table
print("\nSummary")
print("-----------------------------------------")
print("Learning Rate\tEpochs\tTest Accuracy")
print("-----------------------------------------")

for key, value in results.items():
    print(f"{key[0]}\t\t{key[1]}\t{value['accuracy']:.4f}")
```

```python
wine = load_wine()
evaluate_dataset(wine, "Wine Dataset")
```




```
Wine Dataset
Accuracy: 1.0

Confusion Matrix:
[[14  0  0]
```